

1. What is OOP?

Object-Oriented Programming is a programming approach where we organize software around **objects**.

An object generally contains:

- **Data** → attributes/state
- **Functions** → behavior/operations

Example:

```
class Student
{
    int rollNo;
    string name;

public:
    void setData(int r, string n)
    {
        rollNo = r;
        name = n;
    }

    void display()
    {
        cout << rollNo << " " << name;
    }
};
```

Creating an object:

Student s1;

Here:

Student → Class

s1 → Object

rollNo → Data

name → Data

setData → Behavior

display → Behavior

Interview definition

A class is a user-defined data type that combines data and functions into a single unit, while an object is an instance of that class.

2. Class vs Object

This is one of the first interview questions.

Class	Object
Blueprint/template	Instance
Logical entity	Physical/runtime entity
Does not represent an individual object	Represents an individual entity
Defines properties and behavior	Uses those properties and behavior
Example: Student	Example: s1

Example:

```
class Car
```

```
{
```

```
public:
```

```
    string color;
```

```
    void start()
```

```
    {
```

```
        cout << "Car Started";
```

```
    }
```

```
};
```

```
Car c1;
```

```
Car c2;
```

Here Car is the class and c1, c2 are objects.

Think:

Class = building plan

Object = actual building

//self study

1. Operator Overloading ++ — Time Class

```
#include <iostream>
```

```
using namespace std;
```

```
// Class representing Time
```

```
class Time
```

```
{
```

```
    // Private data members
```

```
    int hour;
```

```
    int minute;
```

```
public:
```

```
    // Constructor to initialize hour and minute
```

```
    Time(int h, int m)
```

```
    {
```

```
        hour = h;
```

```
        minute = m;
```

```
    }
```

```
    /*
```

```
        Operator Overloading for ++
```

```
Normally:
```

```
    ++x
```

```
For an object:
```

```
    ++t
```

```
C++ will call:
```

```
    t.operator++()
```

```
Here we increase the minute by 1.
```

```
*/
```

```
void operator++()
```

```
{
```

```
    minute++;
```

```
    // If minutes become 60,
```

```
    // reset minutes to 0 and increase the hour.
```

```
    if (minute == 60)
```

```
    {
```

```
        minute = 0;
```

```
        hour++;
```

```
    }
```

```
}
```

```

// Function to display time
void display()
{
    cout << hour << ":" << minute << endl;
}
};

int main()
{
    // Create a Time object
    // Initial time = 10:59
    Time t(10, 59);

    cout << "Before increment: ";
    t.display();

    /*
        Using overloaded ++ operator

        This:
            ++t

        internally calls:
            t.operator++()
    */
    ++t;

    cout << "After increment: ";
    t.display();

    return 0;
}

```

Output

Before increment: 10:59
After increment: 11:0

Student should observe

```

++t
↓
t.operator++()
↓
minute++
↓
60 minutes?
↓

```

Yes → minute = 0, hour++

2. Operator Overloading - — Vector Class

Here we will create a vector containing **3 elements**.

If:

Vector 1 = 10 20 30

Vector 2 = 2 5 10

Then:

Vector 1 - Vector 2

↓

8 15 20

```
#include <iostream>
```

```
using namespace std;
```

```
// Class representing a Vector
```

```
class Vector
```

```
{
```

```
    // Private array containing 3 elements
```

```
    int a[3];
```

```
public:
```

```
    /*
```

```
        Function to accept vector elements.
```

```
        The user enters 3 values,  
        which are stored inside the array.
```

```
    */
```

```
    void set()
```

```
    {
```

```
        cout << "Enter 3 values: ";
```

```
        for (int i = 0; i < 3; i++)
```

```
        {
```

```
            cin >> a[i];
```

```
        }
```

```
    }
```

/*

Operator Overloading for -

Normally:

$a - b$

For objects:

$v1 - v2$

C++ will call:

$v1.operator-(v2)$

$v1$ is the calling object.

$v2$ is passed as a parameter.

We subtract corresponding elements:

$v1[0] - v2[0]$

$v1[1] - v2[1]$

$v1[2] - v2[2]$

The result is stored in a temporary object.

*/

Vector operator-(Vector v)

{

// Create a temporary Vector object

Vector temp;

// Subtract corresponding elements

for (int i = 0; i < 3; i++)

{

temp.a[i] = a[i] - v.a[i];

}

// Return the resulting Vector object

return temp;

}

/*

Function to display the vector

*/

void display()

{

for (int i = 0; i < 3; i++)

{

cout << a[i] << " ";

}

```

        cout << endl;
    }
};

int main()
{
    // Create three Vector objects
    Vector v1, v2, v3;

    // Accept first vector
    cout << "Vector 1" << endl;
    v1.set();

    // Accept second vector
    cout << "Vector 2" << endl;
    v2.set();

    /*
        Using overloaded - operator

        This:
            v1 - v2

        internally calls:
            v1.operator-(v2)

        The returned object is stored in v3.
    */
    v3 = v1 - v2;

    // Display result
    cout << "Result: ";
    v3.display();

    return 0;
}

```

Sample Input

Vector 1
Enter 3 values: 10 20 30

Vector 2
Enter 3 values: 2 5 10

Sample Output

Result: 8 15 20

★ Key Points for Self-Study

Students should specifically identify these three things:

++ → Unary Operator

`++t;`

Only **one object** is involved.

`void operator++()`

- → Binary Operator

`v3 = v1 - v2;`

Two objects are involved.

`Vector operator-(Vector v)`

Here:

`v1` → calling object

`v2` → parameter

`v3` → result

Most important concept

Member operator overloading:

`v1 - v2`

↓

`v1.operator-(v2)`

Left-side object → Calling object

Right-side object → Parameter